

System Design Challenge — Brief para el Candidato (Cloud Engineer Senior)

Este ejercicio busca conocer **cómo piensas y diseñas soluciones en la nube**, no que memorices servicios ni entregues código perfecto de producción. Queremos ver tu **razonamiento de arquitectura, decisiones de diseño y los tradeoffs** que haces. Un buen diseño con decisiones bien justificadas vale mucho más que un diagrama lleno de cajas.

El reto

 "LiveTix" — Plataforma global de venta de boletos para eventos en vivo

Imagina una startup que vende boletos para conciertos, deportes y teatro **a nivel global**. Tú eres el arquitecto de nube responsable de diseñar la plataforma desde cero.

Contexto del negocio:

- Opera en **4 países** (puedes elegir cuáles) en 3 continentes distintos. Los usuarios esperan que la página cargue rápido sin importar dónde estén.
- El producto estrella son las **ventas de alta demanda**: cuando un artista famoso libera boletos, decenas de miles de personas entran **en el mismo minuto**. El tráfico puede ser **10-20x** el normal, y llega en segundos.
- Cada evento tiene **inventario limitado** (asientos numerados). **Nunca** se puede vender el mismo asiento dos veces, ni bajo altísima concurrencia.
- El pago se procesa con una **pasarela externa** (tipo Stripe/PayPal). La plataforma no guarda tarjetas.
- Regla de oro: **jamás cobrar a un cliente sin entregarle su boleto**. La conciliación pago ↔ boleto debe ser impecable.
- El equipo es pequeño (5 ingenieros); la solución debe ser **operable** sin un ejército de gente.

Qué entregar

No hay una respuesta única correcta. Nos interesa tu criterio.

1. Diagrama de arquitectura

Un diagrama (herramienta libre: draw.io, Excalidraw, Miro) que muestre:

- Los componentes principales y cómo se comunican
- Cómo fluye una compra de boleto de inicio a fin
- Qué es multi-región y qué es global
- Dónde vive el dato y de qué tipo es cada almacén

Puedes usar **cualquier nube** (AWS, GCP, Azure) o combinación. Si eliges multi-nube, explica por qué.

2. Documento de decisiones (2-4 páginas)

Explica tus **decisiones y tradeoffs**. Nos importa más el *por qué* que el *qué*. Cubre al menos:

- **Manejo de picos:** ¿Cómo garantizas que el checkout no se caiga cuando llegan 20x usuarios en un minuto? (el autoescalado reactivo tarda minutos en responder)
- **Concurrencia de inventario:** ¿Cómo evitas vender el mismo asiento dos veces bajo alta concurrencia?
- **Consistencia de pago:** ¿Cómo garantizas que nunca haya un cobro sin boleto entregado, aun si la pasarela o tu servicio fallan a la mitad?
- **Multi-región:** ¿Cómo das baja latencia por país controlando costos? ¿Qué replicas y qué no?
- **Confiabilidad:** ¿Qué pasa si se cae una zona o región completa? ¿El cliente lo nota?
- **Observabilidad:** ¿Cómo sabes que algo falla antes de que te llamen los usuarios?

3. Infraestructura como Código (conceptual)

✓ **No necesitas escribir Terraform funcional.** Solo describe el concepto.

- ¿Cómo organizarías el código de infraestructura? (módulos, ambientes, versionado)
- ¿Cómo manejarías secretos y credenciales?
- ¿Cómo sería tu pipeline de despliegue (CI/CD)?

Puedes hacerlo en prosa, pseudocódigo o un diagrama de estructura de carpetas. Lo que mejor comunique tu idea.

Cómo lo vamos a evaluar

Buscamos que:

- **Justifiques cada decisión** con una razón (costo, escala, consistencia, resiliencia) — no que solo nombres tecnologías
- Menciones **qué sacrificas** con cada elección (todo diseño tiene tradeoffs)
- Seas honesto sobre lo que **no** alcanzaste a resolver y qué harías con más tiempo
- Diseñes algo que un equipo pequeño **pueda operar** de verdad

No buscamos: que uses la tecnología de moda, un diseño perfecto/exhaustivo, ni que cubras cada caso extremo — prioriza y dinos por qué.

Formato y tiempo

- **Esfuerzo esperado:** 4-6 horas. No queremos que dediques tu semana.

- **Plazo:** [X días] desde que recibes este brief.
- **Entrega:** un repositorio Git o un PDF + el diagrama. Como prefieras